DATASTOCK · REGISTRO DE PRUEBAS

UT1 · Ficheros y formatos de intercambio de datos | Acceso a Datos | 2.º DAM | IES Isidra de Guzmán | Curso 2026–2027

Alumno/a	Edicxon Gabriel López Duque
Repositorio GitHub	https://github.com/edicxonlopezcolab/datastock
Etiqueta de entrega	ut1-entrega (último commit de código: 69abef6)
Fecha de ejecución	Del 20 al 28 de octubre de 2026
Resultado global	**52 pruebas ejecutadas · 52 superadas · 0 no superadas · 0 no ejecutadas**

Cada prueba indica los datos de entrada, el resultado esperado y el obtenido. La salida de consola de cada una se guarda en evidencias/rfXX/ con el identificador de la prueba como nombre de fichero.

1. Resumen de pruebas

1.1. Por requisito

Requisito	Descripción	Pruebas	Superadas	Evidencias	Commit
RF-FIC-01	Explorar rutas, ficheros y directorios	7	**7**	evidencias/rf01/	52e6f26
RF-FIC-02	Importar catálogo JSON	6	**6**	evidencias/rf02/	1d451a8
RF-FIC-03	Importar catálogo XML mediante DOM	6	**6**	evidencias/rf03/	4f5db51
RF-FIC-04	Consultar y buscar productos en memoria	8	**8**	evidencias/rf04/	8bca4a9
RF-FIC-05	Exportar el inventario a JSON y XML	7	**7**	evidencias/rf05/	f8b0603
RF-FIC-06	Guardar y recuperar el inventario (serialización)	4	**4**	evidencias/rf06/	cf616c1
RF-FIC-07	Consultas XML mediante XPath (independiente)	7	**7**	evidencias/rf07/	1efdeb3
RF-FIC-08	Acceso aleatorio con RandomAccessFile (independiente)	7	**7**	evidencias/rf08/	69abef6
Total		**52**	**52**		

1.2. Detalle de las pruebas

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
RF-FIC-01 · Explorar rutas, ficheros y directorios				
CP-FIC-01	datos/pruebas/rf01	Listar ficheros y directorios con tamaño y totales.	[DIR] antiguos, 2 ficheros de 35 bytes; Total ficheros: 2 | Directorios: 1	**SUPERADA**
CP-FIC-02	datos/no_existe	Indicar que la ruta no existe mostrando la absoluta.	La ruta no existe: ...\datos\no_existe; vuelve al menú sin cerrarse.	**SUPERADA**
CP-FIC-03	pom.xml	Mostrar nombre, ruta absoluta y tamaño en bytes.	Nombre pom.xml, ruta absoluta y 1297 bytes.	**SUPERADA**
CP-FIC-04	Intro vacío	Indicar que la ruta está vacía.	Mensaje de error de ruta vacía.	**SUPERADA**
CP-FIC-05	C:\System Volume Information	Capturar la restricción de acceso y avisar.	Mensaje de error de permisos (AccessDeniedException).	**SUPERADA**
CP-FIC-06	datos/pruebas/vacia	Indicar que el directorio está vacío.	Mensaje de "Directorio vacío".	**SUPERADA**
CP-FIC-07	datos/a*b	Manejar caracteres no válidos en la ruta (InvalidPathException).	Muestra el mensaje: Error: La ruta escrita contiene caracteres no válidos.	**SUPERADA**
RF-FIC-02 · Importar catálogo JSON				
CP-JSON-01	datos/pruebas/rf02/tres_validos.json	Se importan los tres	3 encontrados, 3 importados, 0 rechazados; web-012 normalizado a WEB-012	**SUPERADA**
CP-JSON-02	datos/pruebas/rf02/ tres_validos_dos_invalidos.json	Se importan tres y se explican dos rechazos	5 / 3 / 2; SSD-021: precio debe ser mayor que 0; producto 4 (sin código): código obligatorio	**SUPERADA**
CP-JSON-03	datos/ejemplos/catalogo_malformado.json	No se añade ningún producto	Error de JSON mal formado; productos en memoria: 6 antes y después	**SUPERADA**
CP-JSON-04	datos/pruebas/rf02/tres_validos.json importado por segunda vez	Se conserva el existente	3 / 0 / 3; MON-010, ALT-011 y WEB-012 "ya existe en el inventario"; memoria sigue en 6	**SUPERADA**
CP-JSON-05	datos/pruebas/rf02/reglas_validacion.json	Rechazar cada producto con su motivo	10 / 0 / 10; un motivo distinto por regla; código vacío mostrado como "(sin código)"	**SUPERADA**
CP-JSON-06	datos/pruebas/rf02/sin_array.json	Rechazar el documento completo	Error "falta el array productos"; inventario sin cambios	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
RF-FIC-03 · Importar catálogo XML mediante DOM				
CP-XML-01	datos/pruebas/rf03/tres_validos.xml	Se importan tres	3 / 3 / 0; tab-032 normalizado a TAB-032	**SUPERADA**
CP-XML-02	datos/pruebas/rf03/con_invalidos.xml	Se rechazan solo los inválidos	4 / 2 / 2; producto 2 (AUR-041) precio no positivo; producto 4 (REG-043) stock no entero	**SUPERADA**
CP-XML-03	datos/pruebas/rf03/con_dtd.xml (DOCTYPE con entidad externa)	Se rechaza el documento	Error “XML mal formado o con DTD no permitida”; memoria sigue en 7	**SUPERADA**
CP-XML-04	datos/pruebas/rf03/duplicado_json.xml tras importar catalogo_valido.json	Se rechaza el duplicado	2 / 1 / 1; TEC-001 “ya existe en el inventario”; PAD-050 importado	**SUPERADA**
CP-XML-05	datos/pruebas/rf03/malformado.xml	Rechazar el documento	Error “XML mal formado”; memoria sin cambios	**SUPERADA**
CP-XML-06	datos/pruebas/rf03/raiz_incorrecta.xml	Rechazar el documento completo	Error “la raíz debe ser catalogo”; memoria sin cambios	**SUPERADA**
RF-FIC-04 · Consultar y buscar productos en memoria				
CP-CON-01	Opción 4.1 sin importar	Mensaje de inventario vacío	“No hay productos en el inventario.”	**SUPERADA**
CP-CON-02	Opción 4.2, tec-001	Localiza TEC-001	Ficha completa de TEC-001 (Teclado USB, 19.95 €, stock 12)	**SUPERADA**
CP-CON-03	Opción 4.3, teclado	Coincidencias parciales	Encuentra “Teclado USB” sin distinguir mayúsculas	**SUPERADA**
CP-CON-04	Importar catalogo_valido.json y los XML de rf03, y listar	Se ven productos de ambos formatos	8 productos ordenados por código: RAT-002 y TEC-001 (JSON) junto a ESC-031, IMP-030, TAB-032… (XML)	**SUPERADA**
CP-CON-05	Opción 4.1 con 5 productos importados en otro orden	Listado ordenado por código	ALT-011, MON-010, RAT-002, TEC-001, WEB-012; total 5	**SUPERADA**
CP-CON-06	Opción 4.2, XXX-999	Informar de que no existe	“No existe ningún producto con ese código.”	**SUPERADA**
CP-CON-07	Opción 4.2, Intro vacío	Rechazar búsqueda vacía	“El código no puede estar vacío.”	**SUPERADA**
CP-CON-08	Opción 4.3, impresora	Informar de cero coincidencias	“No hay coincidencias.”	**SUPERADA**
RF-FIC-05 · Exportar el inventario a JSON y XML				
CP-EXP-01	3 productos de tres_validos.json, exportar a datos/salidas/tres.json	JSON válido con tres productos	[OK] 3 productos; raíz productos, ordenados por código, UTF-8	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
CP-EXP-02	Mismos 3 productos, exportar a datos/salidas/tres.xml	XML válido con tres productos	[OK] 3 productos; raíz catalogo, indentado, UTF-8	**SUPERADA**
CP-EXP-03	Exportar a un fichero existente y responder N	Fichero original intacto	“Exportación cancelada.”; el fichero no se modifica	**SUPERADA**
CP-EXP-04	Importar tres_validos.json y exportar a XML	Se conservan los datos	El XML contiene los tres productos con los mismos códigos, nombres, precios, stock e IDs	**SUPERADA**
CP-EXP-05	Reiniciar e importar tres.xml y tres.json en inventario vacío	Se recuperan los mismos productos	XML: 3 / 3 / 0; JSON: 3 / 0 / 3 (mismos códigos); listado idéntico al original	**SUPERADA**
CP-EXP-06	Exportar con el inventario vacío	No se genera fichero	“No hay productos para exportar.”	**SUPERADA**
CP-EXP-07	Exportar a carpeta_que_no_existe/x.json	Rechazar el destino	“[ERROR] la carpeta de destino no existe”	**SUPERADA**
RF-FIC-06 · Guardar y recuperar el inventario (serialización)				
CP-SER-01	Guardar 5 productos en datos/salidas/inventario.dat, cerrar y recuperar	Se recuperan los cinco	[OK] 5 guardados; tras reiniciar, [OK] 5 recuperados; listado con ALT-011, MON-010, RAT-002, TEC-001 y WEB-012	**SUPERADA**
CP-SER-02	Recuperar inventario.dat con 3 productos en memoria y responder N	Se conserva el inventario actual	“Recuperación cancelada.”; siguen los 3 productos	**SUPERADA**
CP-SER-03	datos/pruebas/rf06/danado.dat (texto, no serializado)	Se informa y no cambia la memoria	“[ERROR] no se pudo recuperar el inventario…”; la memoria sigue con 3 productos	**SUPERADA**
CP-SER-04	Guardar con el inventario vacío	No se crea fichero	“No hay productos para guardar.”; no se llega a pedir la ruta	**SUPERADA**
RF-FIC-07 · Consultas XML mediante XPath (independiente)				
CP-XPATH-01	catalogo_xpath.xml, código tec-001	Se encuentra el producto	TEC-001 Teclado USB 19.95 €; comparación en Java sin distinguir mayúsculas	**SUPERADA**
CP-XPATH-02	Categoría 1	Solo productos de categoría 1	TEC-001 y RAT-002; total 2	**SUPERADA**
CP-XPATH-03	Precio superior a 100	Excluye exactamente 100	Solo MON-003 (149.99); DIS-004 (100.00) no aparece	**SUPERADA**
CP-XPATH-04	Contar en catalogo_xpath.xml	Cuenta todos, aunque haya datos incorrectos	5, incluido ERR-005 (precio “abc”, stock -3)	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
CP-XPATH-05	datos/pruebas/rf03/con_dtd.xml	Se rechaza	“[ERROR] … contiene DOCTYPE (no permitido)”; el programa termina sin abrir el menú	**SUPERADA**
CP-XPATH-06	Contar en datos/pruebas/rf03/tres_validos.xml	Resultado 3	“Productos en el documento: 3”	**SUPERADA**
CP-XPATH-07	Código XXX-999; categoría abc y 0	Mensajes sin cerrar la aplicación	“No se ha encontrado…”, “debe ser un número entero válido”, “debe ser mayor que 0”	**SUPERADA**
RF-FIC-08 · Acceso aleatorio con RandomAccessFile (independiente)				
CP-RAF-01	cuatro.dat (4 productos), generar datos/salidas/stock.bin	Fichero de 144 bytes	“Registros: 4 | Tamaño: 144 bytes”	**SUPERADA**
CP-RAF-02	Consultar posición 2	Acceso directo al tercer registro	seek(72): MON-003 | 149.99 € | stock 8	**SUPERADA**
CP-RAF-03	Modificar stock de la posición 2 a 15	Solo cambia ese campo	“stock 8 → 15”; en el listado posterior los otros tres registros no cambian	**SUPERADA**
CP-RAF-04	Consultar posición 4 en un fichero de 4 registros	Se rechaza	“posición fuera de rango”	**SUPERADA**
CP-RAF-05	datos/pruebas/rf08/stock_malo.bin (26 bytes)	Se informa de fichero inválido	“fichero inválido: 26 bytes no es múltiplo de 36”	**SUPERADA**
CP-RAF-06	Nuevo stock -5 en la posición 1	Se rechaza	“el stock no puede ser negativo”; el registro no cambia	**SUPERADA**
CP-RAF-07	Generar sin cargar productos	No se genera el fichero	“no hay productos en memoria: no se genera el fichero”	**SUPERADA**

Estados posibles: SUPERADA / NO SUPERADA / NO EJECUTADA. Las discrepancias encontradas durante las pruebas y su corrección se explican en el apartado 3.

2. Fichas detalladas de prueba

Una ficha por requisito, con la prueba más representativa de cada uno.

CP-FIC-01 · RF-FIC-01 · Explorar rutas del sistema de archivos	
Objetivo	Comprobar que se identifica correctamente si una ruta es un directorio, listando sus hijos inmediatos, tamaños y totales.
Precondiciones	Tener creada la estructura de pruebas datos/pruebas/rf01/ con dos ficheros .txt y una subcarpeta.
Datos de entrada	datos/pruebas/rf01
Pasos ejecutados	1. Iniciar la aplicación y seleccionar la opción 1 (Explorar ficheros y directorios). 2. Introducir la ruta del directorio de pruebas. 3. Pulsar Intro.
Resultado esperado	Se muestran los directorios con la etiqueta [DIR], los ficheros con su tamaño en bytes y un resumen final con el total de ficheros y directorios.
Resultado obtenido	Se listó correctamente [DIR] antiguos, los dos ficheros de 35 bytes y el resumen Total ficheros: 2 | Directorios: 1.
Estado	**SUPERADA**
Evidencia	evidencias/rf01/CP-FIC-01_directorio.txt · commit 52e6f26

CP-JSON-02 · RF-FIC-02 · Importar un catálogo JSON	
Objetivo	Comprobar que, en un fichero con productos válidos e inválidos, se importan solo los válidos y se explica el motivo de cada rechazo.
Precondiciones	Inventario con los productos de CP-JSON-01 (3 en memoria). Fichero datos/pruebas/rf02/tres_validos_dos_invalidos.json con cinco productos: tres correctos, uno con precio negativo y uno sin código.
Datos de entrada	datos/pruebas/rf02/tres_validos_dos_invalidos.json
Pasos ejecutados	1. Seleccionar la opción 2 (Importar productos desde JSON). 2. Introducir la ruta del fichero. 3. Revisar el resumen y el contador de productos en memoria.
Resultado esperado	5 encontrados, 3 importados y 2 rechazados, con el motivo de cada rechazo. Los productos en memoria pasan de 3 a 6.
Resultado obtenido	Encontrados: 5, Importados: 3, Rechazados: 2. Incidencias: “Producto 2 (SSD-021): el precio debe ser mayor que 0” y “Producto 4 (sin código): el código es obligatorio”. Productos en memoria: 6.
Estado	**SUPERADA**
Evidencia	evidencias/rf02/CP-JSON-02_tres_validos_dos_invalidos.txt · commit 1d451a8

CP-XML-03 · RF-FIC-03 · Importar un catálogo XML mediante DOM	
Objetivo	Comprobar que el analizador rechaza un documento con DTD y entidades externas (protección frente a XXE) sin modificar el inventario.
Precondiciones	Inventario con 7 productos cargados en las pruebas anteriores. Fichero datos/pruebas/rf03/con_dtd.xml con una declaración DOCTYPE que define una entidad externa apuntando a un fichero del sistema (file:///C:/Windows/win.ini).
Datos de entrada	datos/pruebas/rf03/con_dtd.xml
Pasos ejecutados	1. Seleccionar la opción 3 (Importar productos desde XML). 2. Introducir la ruta del fichero. 3. Revisar el mensaje y el contador de productos en memoria.
Resultado esperado	El documento se rechaza completo por contener DOCTYPE; no se lee el fichero externo ni se añade ningún producto.
Resultado obtenido	“[ERROR] el fichero contiene un XML mal formado o con DTD no permitida”. Productos en memoria: 7 antes y después. No aparece ningún mensaje técnico del analizador por consola.
Estado	**SUPERADA**
Evidencia	evidencias/rf03/CP-XML-03_con_dtd.txt · commit 4f5db51

CP-CON-02 · RF-FIC-04 · Consultar y buscar productos en memoria	
Objetivo	Comprobar que la búsqueda por código encuentra el producto aunque el usuario escriba el código en minúsculas.
Precondiciones	Inventario con 5 productos importados desde catalogo_valido.json y tres_validos.json, entre ellos TEC-001.
Datos de entrada	tec-001
Pasos ejecutados	1. Seleccionar la opción 4 (Consultar productos). 2. Seleccionar 2 (Buscar producto por código). 3. Introducir tec-001.
Resultado esperado	Se muestra la ficha completa de TEC-001, sin distinguir mayúsculas.
Resultado obtenido	Se muestra Código TEC-001, Nombre Teclado USB, Precio 19.95 €, Stock 12, Categoría 1, Proveedor 1.
Estado	**SUPERADA**
Evidencia	evidencias/rf04/CP-CON-02_buscar_tec-001.txt · commit 8bca4a9

CP-EXP-05 · RF-FIC-05 · Exportar el inventario a JSON y XML	
Objetivo	Comprobar que los ficheros exportados pueden volver a importarse en un inventario vacío y que se recuperan los mismos productos.
Precondiciones	Ficheros datos/salidas/tres.json y datos/salidas/tres.xml generados en CP-EXP-01 y CP-EXP-02 a partir de tres_validos.json. Aplicación reiniciada, con 0 productos en memoria.
Datos de entrada	datos/salidas/tres.xml y datos/salidas/tres.json
Pasos ejecutados	1. Iniciar la aplicación. 2. Opción 3 con tres.xml. 3. Opción 2 con tres.json. 4. Opción 4 → 1 (listar).
Resultado esperado	El XML importa los tres productos; el JSON los reconoce como ya existentes, lo que demuestra que ambos ficheros contienen los mismos productos; el listado coincide con los datos originales.
Resultado obtenido	XML: 3 encontrados, 3 importados, 0 rechazados. JSON: 3 encontrados, 0 importados, 3 rechazados por “ya existe en el inventario” (ALT-011, MON-010, WEB-012). El listado muestra los tres productos con los mismos nombres, precios, stock e IDs que en tres_validos.json.
Estado	**SUPERADA**
Evidencia	evidencias/rf05/CP-EXP-05_reimportar_inventario_vacio.txt, CP-EXP-01_tres.json y CP-EXP-02_tres.xml · commit f8b0603

CP-SER-03 · RF-FIC-06 · Guardar y recuperar el inventario mediante serialización	
Objetivo	Comprobar que, al intentar recuperar un fichero dañado o que no es un inventario serializado, se informa del error y el inventario en memoria no se modifica.
Precondiciones	Aplicación iniciada con 3 productos en memoria, importados desde datos/pruebas/rf03/tres_validos.xml. Fichero datos/pruebas/rf06/danado.dat que contiene texto plano en lugar de objetos serializados.
Datos de entrada	datos/pruebas/rf06/danado.dat
Pasos ejecutados	1. Seleccionar la opción 6 (Guardar y recuperar inventario). 2. Seleccionar 2 (Recuperar inventario desde fichero .dat). 3. Introducir la ruta del fichero dañado. 4. Intentar a continuación recuperar datos/salidas/inventario.dat para comprobar cuántos productos hay en memoria.
Resultado esperado	Se rechaza el fichero con un mensaje comprensible, no se pide confirmación y el inventario sigue con los 3 productos.
Resultado obtenido	“[ERROR] no se pudo recuperar el inventario: el fichero está dañado o no es un inventario de DataStock”. En la operación siguiente, el mensaje “El inventario actual tiene 3 productos” confirma que la memoria no cambió.
Estado	**SUPERADA**
Evidencia	evidencias/rf06/CP-SER-03_fichero_danado.txt · commit cf616c1

CP-XPATH-03 · RF-FIC-07 · Consultas XML mediante XPath	
Objetivo	Comprobar que el filtro de precio superior excluye los productos cuyo precio es exactamente el valor indicado.
Precondiciones	Fichero datos/pruebas/rf07/catalogo_xpath.xml con cinco productos, entre ellos MON-003 (149.99) y DIS-004 (exactamente 100.00).
Datos de entrada	fichero catalogo_xpath.xml; precio mínimo 100
Pasos ejecutados	1. Ejecutar mvn exec:java -Dexec.mainClass=es.iesisidra.datastock.complementarias.XPathActividad. 2. Introducir la ruta del fichero. 3. Elegir la opción 3 (Precio superior). 4. Introducir 100.
Resultado esperado	Solo aparecen los productos con precio estrictamente mayor que 100; DIS-004 no aparece.
Resultado obtenido	“MON-003 | Monitor 24 pulgadas | 149.99 €”; Total encontrados: 1. DIS-004 no aparece. La expresión usada es /catalogo/producto[precio > 100], con el valor validado previamente como BigDecimal.
Estado	**SUPERADA**
Evidencia	evidencias/rf07/CP-XPATH-03_precio_superior.txt · commit 1efdeb3

CP-RAF-03 · RF-FIC-08 · Acceso aleatorio a un fichero binario de stock	
Objetivo	Comprobar que se modifica solo el campo stock de un registro, accediendo directamente a su posición sin reescribir el resto.
Precondiciones	datos/salidas/stock.bin generado desde datos/pruebas/rf08/cuatro.dat, con 4 registros (144 bytes). En la posición 2 está MON-003 con stock 8.
Datos de entrada	posición 2, nuevo stock 15
Pasos ejecutados	1. Ejecutar mvn exec:java "-Dexec.mainClass=es.iesisidra.datastock.complementarias.AccesoAleatorioActividad". 2. Cargar cuatro.dat e indicar stock.bin. 3. Opción 4, posición 2, nuevo stock 15. 4. Opción 2 para listar.
Resultado esperado	El stock de MON-003 pasa de 8 a 15; código, precio y los demás registros no cambian.
Resultado obtenido	“Actualización correcta: stock 8 → 15”. El listado muestra MON-003 con stock 15 y ALT-011 (14), CAB-005 (40) y TEC-001 (12) sin cambios. Se escribió con seek(2 × 36 + 32) = seek(104) y writeInt, volviendo a posicionar después de leer el valor anterior.
Estado	**SUPERADA**
Evidencia	evidencias/rf08/CP-RAF-03_modificar_stock.txt · commit 69abef6

3. Incidencias detectadas y correcciones

Problemas encontrados al ejecutar las pruebas o al revisar el código, y cómo se corrigieron. Todas las pruebas afectadas se repitieron tras la corrección.

ID / prueba	Problema detectado	Corrección aplicada	Resultado tras repetir
RF-FIC-01 (listado de directorio)	Detectado en revisión de código: si fallaba Files.size se mostraba 0 bytes, un dato falso	Se muestra “tamaño no disponible” en lugar de un valor inventado	Listado correcto; sin datos falsos
RF-FIC-02 (incidencias de importación)	Detectado en revisión de código: un producto sin campo codigo mostraba “Producto N (null)”	Variable codigoMostrar solo para el mensaje, que muestra “sin código”; el dato original no se modifica	El resumen muestra “(sin código)”
CP-JSON-05	Detectado al ejecutar la prueba: con código vacío ("") la incidencia mostraba “Producto 1 ()”	codigoMostrar trata igual el código nulo y el vacío	Muestra “Producto 1 (sin código): el código es obligatorio”
CP-CON-05	Al ejecutar con el botón Run de VS Code, el símbolo € aparecía como “?” (consola de Windows sin UTF-8)	Ejecutar chcp 65001 y lanzar con mvn exec:java en la misma terminal; no requiere cambios en el código	El precio se muestra con €, p. ej. “19.95 €”

ID / prueba	Problema detectado	Corrección aplicada	Resultado tras repetir
CP-XML-02	La posición del producto salía como “Producto 11” y “Producto 31”: "Producto " + i + 1 concatenaba texto en lugar de sumar	Paréntesis para sumar antes de concatenar: "Producto " + (i + 1)	Muestra “Producto 2” y “Producto 4”
CP-XML-03 y CP-XML-05	El analizador XML imprimía por su cuenta “[Fatal Error] …” en inglés, además del mensaje propio	Se asigna al analizador un DefaultHandler, que no escribe en consola; el error se gestiona solo en el catch	Solo aparece el mensaje “[ERROR] …” de DataStock
RF-FIC-05 (exportar JSON)	Detectado en revisión de código: si el inventario estaba vacío o pedirRutaDestino() devolvía null, el método seguía ejecutándose y habría intentado exportar a un destino nulo (NullPointerException, cierre de la aplicación)	Se añade return tras cada aviso, de modo que el método termina en cuanto no puede exportar	CP-EXP-06 y CP-EXP-07 informan sin cerrar la aplicación
CP-SER-03	Con un fichero dañado se mostraba el mensaje técnico de Java “invalid stream header: 6573746F”, incomprensible para el usuario	Mensaje propio: “no se pudo recuperar el inventario: el fichero está dañado o no es un inventario de DataStock”	Se muestra el mensaje en español; la memoria no cambia
RF-FIC-07 (ejecución)	El comando mvn exec:java -Dexec.mainClass=… del README lanzaba siempre el menú principal: el pom.xml fijaba mainClass en la configuración del plugin, que tiene prioridad sobre la propiedad de la línea de comandos	Se define la propiedad exec.mainClass (por defecto App) y el plugin la usa como ${exec.mainClass}	mvn exec:java abre el menú y -Dexec.mainClass=… abre la actividad indicada
RF-FIC-08 (ejecución en Windows)	En PowerShell, mvn exec:java -Dexec.mainClass=… daba “Unknown lifecycle phase”: PowerShell parte el parámetro por el punto	El parámetro se escribe entre comillas: "-Dexec.mainClass=…"; se documenta en el README	La actividad arranca correctamente en Windows y en Linux

4. Conclusión de la ejecución

Ejecutadas	Superadas	No superadas	No ejecutadas
52	**52**	**0**	**0**

Funcionalidades y observaciones

RF-FIC-01	7 pruebas superadas. Se controlan rutas vacías, inexistentes, con caracteres no válidos, sin permisos y directorios vacíos.

RF-FIC-02	6 pruebas superadas. Se importan los productos válidos y se rechazan, con su motivo, los que incumplen alguna regla de validación, los códigos repetidos en el mismo fichero y los que ya existen en el inventario. Un JSON mal formado o sin el array productos se rechaza entero sin modificar el inventario.
RF-FIC-03	6 pruebas superadas. El XML se analiza con DOM prohibiendo DOCTYPE, lo que rechaza DTD y entidades externas (protección XXE). Se aplican exactamente las mismas validaciones que en JSON y se detectan duplicados respecto a productos importados desde JSON.
RF-FIC-04	8 pruebas superadas. El listado se muestra ordenado por código sin modificar el inventario, y las búsquedas por código y por nombre no distinguen mayúsculas. Se informa del inventario vacío, del código inexistente, del texto vacío y de la ausencia de coincidencias. Los productos importados desde JSON y desde XML se consultan juntos.
RF-FIC-05	7 pruebas superadas. El inventario se exporta ordenado por código a JSON (Gson) y a XML (DOM con Transformer), en UTF-8 e indentado. Se comprueba que existe la carpeta de destino y se pide confirmación antes de sobrescribir. Los ficheros exportados se vuelven a importar sin pérdida de datos.
RF-FIC-06	4 pruebas superadas. El inventario se guarda en un fichero .dat con ObjectOutputStream y se recupera con ObjectInputStream. Antes de sustituir se comprueba que el contenido es una lista de productos válidos y se pide confirmación si hay productos en memoria; el inventario solo se sustituye si la recuperación se completa. Importar JSON/XML añade productos; recuperar un .dat sustituye el inventario.
RF-FIC-07	7 pruebas superadas. XPathActividad abre el XML con la misma protección anti-XXE y permite varias consultas sobre el documento: por código, por categoría, por precio superior y contar. El código del usuario se compara en Java, sin concatenarlo en la expresión; la categoría y el precio se validan como números. Se corrigió el pom.xml para poder lanzar las actividades con -Dexec.mainClass.
RF-FIC-08	7 pruebas superadas. AccesoAleatorioActividad carga los productos desde un .dat de RF-FIC-06, genera stock.bin con registros de 36 bytes truncando el fichero, y permite listar secuencialmente, consultar con seek(posición × 36) y modificar solo el stock con seek(posición × 36 + 32). Se rechazan tamaños no múltiplos de 36, posiciones fuera de rango y stocks negativos. stock.bin es una copia independiente del inventario.

Todos los requisitos de la UT1 están implementados y probados. Funcionalidades pendientes: ninguna.

Fecha: 28 de octubre de 2026